Ministère de la Transition Numérique et de l'Innovation Technologique

Année académique : 2026-2027

ESATIC
ECOLE SUPERIEURE AFRICAINE DES TIC

RAPPORT DE TRAVAUX PRATIQUES

MODULE : CRYPTOGRAPHIE

CONCEPTION ET DÉVELOPPEMENT D'UNE APPLICATION WEB DE CHIFFREMENT DE FICHIERS PAR RSA ENTRE BINÔMES : RSA FILESECURE

Réalisé par :

ASSALÉ N'DAH JEAN-FRANÇOIS

[NOM ET PRÉNOMS DU BINÔME]

Remis le : **[date de remise]**

Enseignant du module

[Nom de l'enseignant]

Enseignant, ESATIC

Application en ligne

rsa-filesecure.vercel.app

Code source : dossier TP-RSA

Sommaire

1. Introduction 3
2. Présentation du projet 3
2.1. Sujet et rôles 3
2.2. Solutions existantes 3
2.3. Cahier des charges 4
3. Notions de cryptographie utilisées 4
3.1. Le chiffrement RSA 4
3.2. Pourquoi RSA ne s'utilise pas seul 5
3.3. Le chiffrement hybride et les autres primitives 5
4. Conception 6
4.1. Architecture 6
4.2. Déroulement d'un échange 6
4.3. Cas d'utilisation 7
4.4. Séquence d'un chiffrement 8
4.5. Choix cryptographiques 8
4.6. Format du fichier chiffré 9
4.7. Structures de données 9
5. Réalisation 10
5.1. Technologies et organisation du code 10
5.2. Interface 10
5.3. Sécurité et déploiement 12
6. Tests et résultats 13
6.1. Tests automatisés 13
6.2. Performances 14
7. Difficultés rencontrées 15
8. Organisation du travail 15
9. Limites et perspectives 16
10. Conclusion 17
Références 18
Annexe A : liste des tests automatisés 19
Annexe B : structure du fichier .rsaenc 19
Annexe C : vérification avec OpenSSL 20

1. Introduction

Rapports, pièces justificatives et données personnelles circulent chaque jour par courrier électronique, par messagerie ou sur des clés USB. Aucun de ces canaux ne garantit que seul le destinataire prévu pourra lire le fichier, ni qu'il n'aura pas été modifié en route. La cryptographie à clé publique répond à ce besoin, et RSA, publié en 1978 par Rivest, Shamir et Adleman, en reste la référence.

Ce projet a été réalisé en binôme dans le cadre du module de cryptographie. L'énoncé était le suivant : « Créer un site web permettant de chiffrer des fichiers à l'aide de l'algorithme RSA, puis de permettre à son binôme de les déchiffrer à l'aide du même algorithme. »

Une difficulté apparaît dès la lecture du sujet : RSA ne chiffre qu'un bloc de quelques centaines d'octets, alors qu'un fichier en compte souvent des millions. Nous y avons répondu avec RSA FileSecure, une application web qui combine RSA et AES dans un chiffrement hybride, et dont tous les calculs s'effectuent dans le navigateur. Ce rapport présente le projet et son cahier des charges, les notions de cryptographie utilisées, la conception, la réalisation, puis les tests, les difficultés rencontrées et les perspectives.

2. Présentation du projet

2.1. Sujet et rôles

Le sujet impose deux rôles. Dans tout le rapport, **Awa** désigne l'expéditrice, qui chiffre un fichier, et **Koffi** le destinataire, qui doit être le seul à pouvoir le lire ; ce sont aussi les noms utilisés dans l'application. L'énoncé laissait ouvertes plusieurs questions : l'échange des clés, la taille des fichiers, la protection de la clé privée et la réaction face à un fichier altéré. Nous les avons traitées comme des exigences, afin de produire un outil réellement utilisable.

2.2. Solutions existantes

Avant de concevoir notre solution, nous avons examiné les outils couramment utilisés pour protéger un fichier avant de l'envoyer.

Tableau 1 : Comparaison des solutions existantes

Solution	Atouts	Limites au regard du projet
GnuPG (OpenPGP)	Chiffrement hybride robuste, gestion des clés	Installation et prise en main techniques
OpenSSL	Boîte à outils complète, référence du domaine	Chiffrement de fichiers à assembler à la main, sans mode authentifié

Solution	Atouts	Limites au regard du projet
Archives chiffrées (7-Zip)	Simple et très répandu	Mot de passe à transmettre par un autre canal ; pas de RSA
Services en ligne	Aucune installation	Le fichier est envoyé sur le serveur d'un tiers

Aucun de ces outils ne réunit simplicité, absence de secret partagé et confidentialité vis-à-vis d'un serveur, et aucun ne montre ce qui se passe pendant le chiffrement, ce qui ne sert pas l'objectif pédagogique du projet.

2.3. Cahier des charges

Nous nous sommes fixé quatre objectifs : chiffrer n'importe quel type de fichier avec RSA au cœur du mécanisme ; garantir que seul le destinataire peut le lire et qu'une modification est détectée ; ne rien confier à un serveur ; rendre chaque étape visible pour l'expliquer pendant la soutenance. Ils se traduisent par les exigences suivantes, chacune rattachée à sa vérification.

Tableau 2 : Exigences et moyens de vérification

Réf.	Exigence	Vérification
EF1	Générer une paire RSA de 2048, 3072 ou 4096 bits	Test de génération
EF2	Exporter la clé publique dans un format standard	Test PEM, deux tests OpenSSL
EF3	Sauvegarder la clé privée chiffrée par un mot de passe	Tests bon et mauvais mot de passe
EF4	Chiffrer tout type de fichier jusqu'à 100 Mo	Tests fichier vide, 300 Ko, plus de 100 Mo
EF5	Restituer le fichier d'origine avec son nom	Test d'aller-retour
EF6	Refuser un fichier modifié, tronqué ou destiné à une autre clé	Six tests de refus
ENF1	N'envoyer aucune donnée sur le réseau	Politique de sécurité du contenu
ENF2	N'utiliser que des algorithmes standards	Interopérabilité avec OpenSSL
ENF3	Rester utilisable sur ordinateur et sur téléphone	Vérification à 375 pixels de large

3. Notions de cryptographie utilisées

3.1. Le chiffrement RSA

RSA est un chiffrement asymétrique : une clé publique chiffre, une clé privée déchiffre. Sa sécurité repose sur la difficulté de factoriser un grand nombre n, produit de deux nombres premiers secrets p et q. On choisit un exposant public e, en pratique 65537, premier avec $\varphi(n) = (p - 1)(q - 1)$, puis l'exposant privé d, inverse de e modulo $\varphi(n)$. La clé publique est le couple (n, e), la clé privée le couple (n, d) :

$$c = m^e \bmod n \qquad m = c^d \bmod n$$

Avec p = 61 et q = 53, on obtient n = 3233 et φ(n) = 3120. Pour e = 17, l'inverse modulaire vaut d = 2753, car 17 × 2753 = 15 × 3120 + 1. Le message 65 se chiffre en 2790, qui se déchiffre bien en 65. Cet exemple est rejoué dans la page « Comprendre RSA » du site et vérifié par les tests.

3.2. Pourquoi RSA ne s'utilise pas seul

Utilisé tel quel, RSA est déterministe, malléable et limité à un bloc plus petit que n. Le remplissage OAEP (RFC 8017) mélange le message avec une valeur aléatoire à l'aide de SHA-256 : le chiffrement devient probabiliste et tout chiffré modifié est rejeté. Il réduit encore la place disponible.

Tableau 3 : Taille maximale d'un message RSA-OAEP avec SHA-256

Taille de clé	Taille de n	Message maximal
2048 bits	256 octets	190 octets
3072 bits	384 octets	318 octets
4096 bits	512 octets	446 octets

Chiffrer 100 Mo bloc par bloc demanderait environ 330 000 opérations RSA. Au déchiffrement, chacune coûte près de 2 millisecondes avec une clé de 3072 bits sur notre machine, soit plus de dix minutes au total.

3.3. Le chiffrement hybride et les autres primitives

La solution standard, adoptée par OpenPGP, combine les deux familles de chiffrement. Une clé de session AES de 256 bits, tirée au hasard pour chaque fichier, chiffre le contenu ; RSA-OAEP ne chiffre que cette clé de 32 octets, avec la clé publique du destinataire. Sans la clé privée de Koffi, la clé de session est inaccessible, et le fichier avec elle : RSA reste le mécanisme qui décide qui peut lire.

Le mode **GCM** (NIST SP 800-38D) fait d'AES un chiffrement authentifié : chaque bloc chiffré porte une étiquette de 128 bits, et la moindre modification fait échouer le déchiffrement. **SHA-256** sert à calculer l'empreinte des clés publiques et à vérifier le fichier restauré. **PBKDF2**, avec 600 000 itérations comme le recommande l'OWASP, dérive du mot de passe la clé qui protège la clé privée.

4. Conception

4.1. Architecture

Le site est une application monopage qui s'exécute entièrement dans le navigateur. Le serveur se contente de livrer des fichiers statiques. Le code est organisé en trois couches : l'interface appelle le moteur cryptographique, qui s'appuie sur l'API Web Crypto du navigateur. Les clés publiques et les fichiers chiffrés circulent par les moyens que choisissent les utilisateurs.

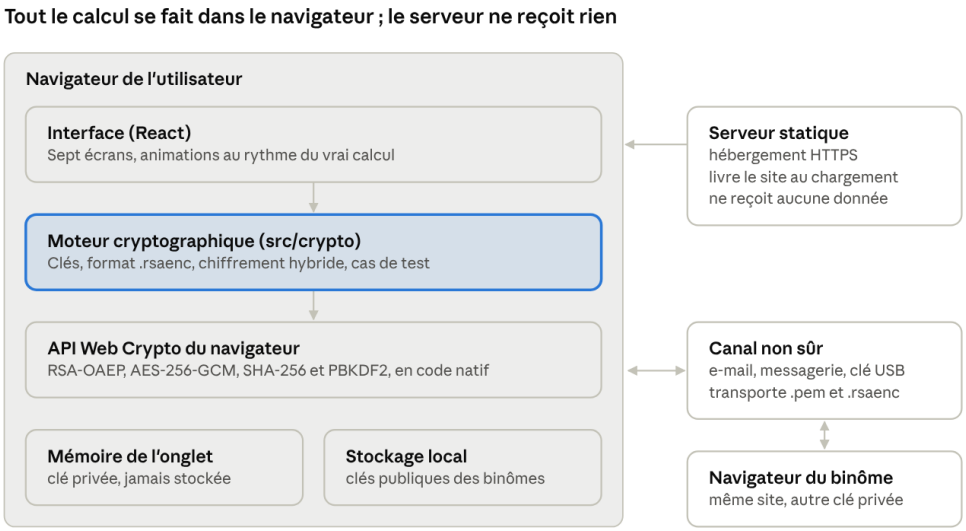


Figure 1 : Architecture de RSA FileSecure

4.2. Déroulement d'un échange

La figure suivante résume la circulation des données entre les deux membres du binôme. La clé privée de Koffi, son mot de passe et la clé de session ne quittent jamais les navigateurs ; le tiers qui écoute le canal ne voit passer qu'une clé publique et un fichier qu'il ne peut ni lire ni modifier sans que cela se voie.

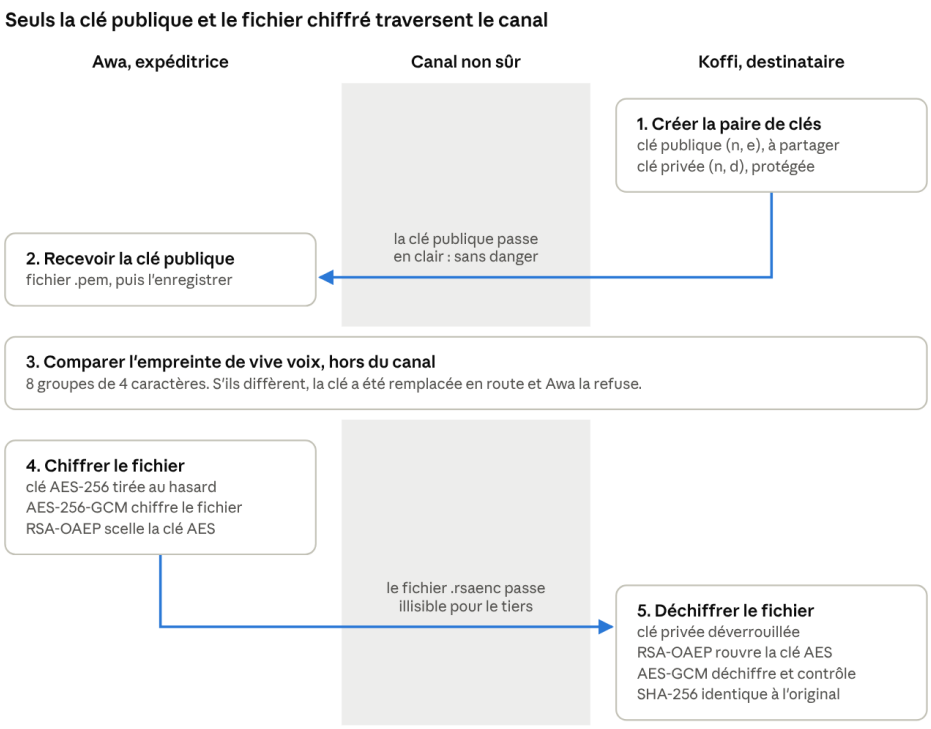


Figure 2 : Déroulement d'un échange entre l'expéditrice et le destinataire

4.3. Cas d'utilisation

Le diagramme suivant présente les services rendus aux trois acteurs. Le chiffrement inclut obligatoirement la vérification de l'empreinte ; le rechargement de la clé privée étend le déchiffrement lorsqu'aucune clé n'est chargée.

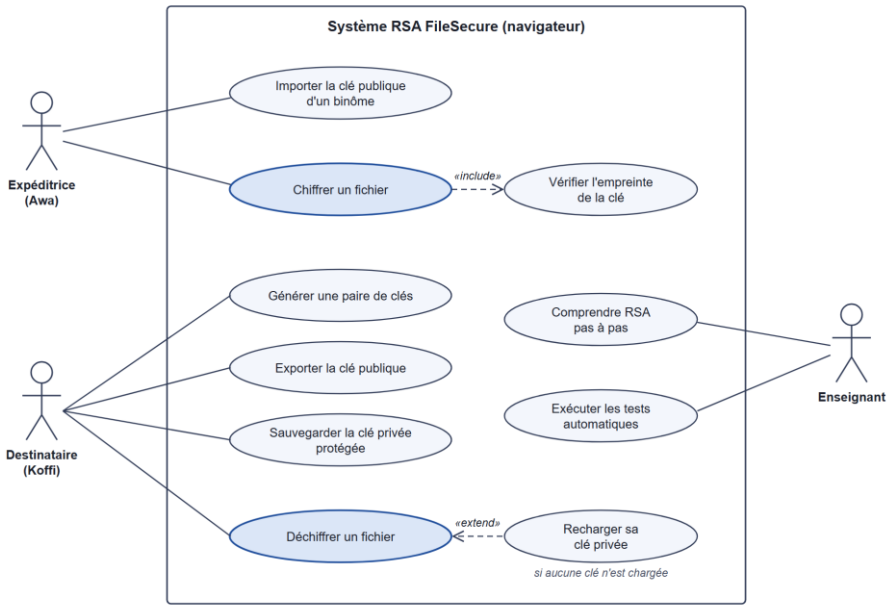


Figure 3 : Diagramme de cas d'utilisation

4.4. Séquence d'un chiffrement

Le diagramme de séquence décrit les échanges entre l'expéditrice, l'interface, le moteur cryptographique et l'API Web Crypto. La clé de session n'existe en clair que dans la mémoire du navigateur, le temps d'être scellée, et l'interface reçoit l'avancement réel du calcul.

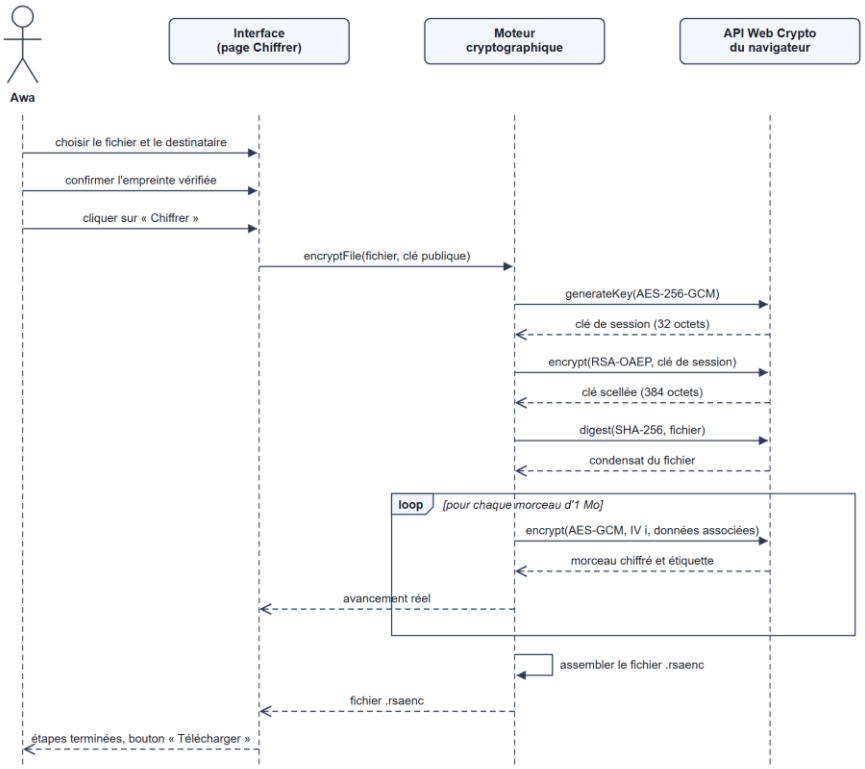


Figure 4 : Diagramme de séquence du chiffrement d'un fichier

4.5. Choix cryptographiques

Tableau 4 : Choix cryptographiques retenus

Élément	Choix retenu	Justification
Clé de session	RSA-OAEP, SHA-256, e = 65537	Remplissage probabiliste ; PKCS #1 v1.5 écarté (attaque de Bleichenbacher, 1998)
Taille de clé	3072 bits par défaut, 2048 au minimum	128 bits de sécurité pour 3072 bits selon le NIST
Contenu	AES-256-GCM, morceaux de 1 Mo	Confidentialité et intégrité, mémoire maîtrisée
Vecteur d'initialisation	8 octets aléatoires et numéro du morceau	Jamais réutilisé pour une même clé
Clé privée	PBKDF2-SHA-256, 600 000 itérations, puis AES-256-GCM	Recommandation de l'OWASP
Empreinte	32 premiers caractères du SHA-256 de la clé publique	Lisible à voix haute en 8 groupes de 4

4.6. Format du fichier chiffré

Le fichier chiffré porte l'extension .rsaenc. Il commence par les six caractères « RSAENC », un octet de version et la longueur de l'en-tête. L'en-tête JSON indique les algorithmes, l'identifiant de la clé du destinataire, la clé de session scellée, le préfixe des vecteurs d'initialisation et le découpage ; viennent ensuite les morceaux chiffrés.

Chaque morceau authentifie, comme données associées, l'en-tête complet, son numéro et un drapeau « dernier morceau ». Modifier l'en-tête, intervertir deux morceaux ou supprimer la fin du fichier fait donc échouer au moins une étiquette. Le nom et le condensat SHA-256 du fichier sont chiffrés avec son contenu : le nom ne circule jamais en clair. Le surcoût reste inférieur à 1 Ko, plus 16 octets par mégaoctet.

4.7. Structures de données

L'application n'utilise pas de base de données. Le diagramme de classes représente les structures manipulées par le moteur cryptographique, telles qu'elles sont typées dans le code : une identité et une clé de binôme partagent les attributs d'une clé publique, le fichier .rsakey protège une identité, et l'en-tête d'un fichier chiffré désigne la clé de son destinataire.

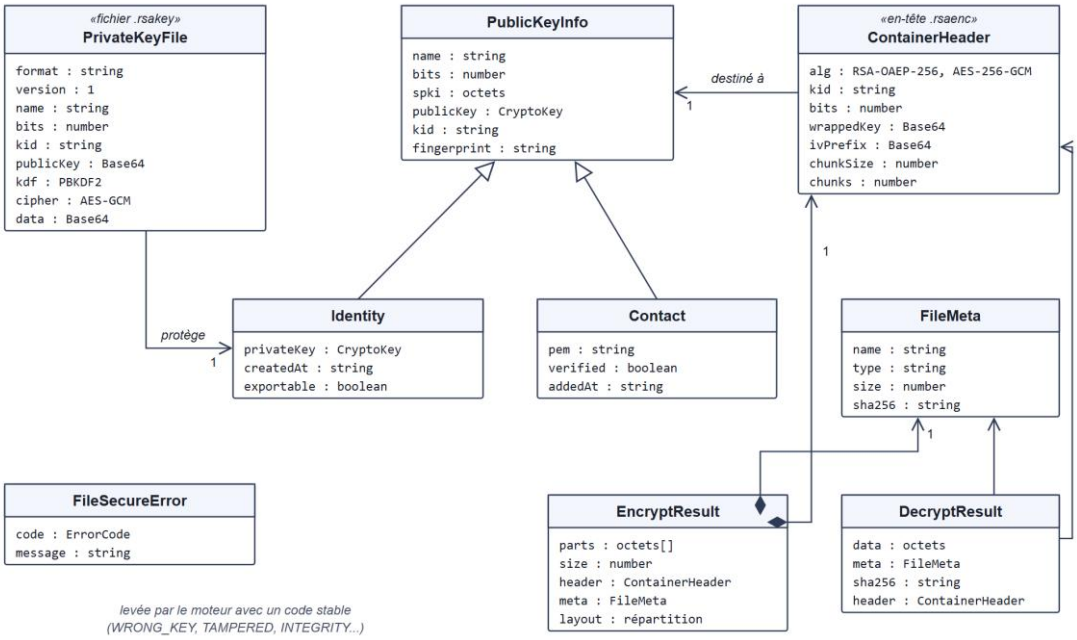


Figure 5 : Diagramme de classes du moteur cryptographique

5. Réalisation

5.1. Technologies et organisation du code

Le site est écrit en TypeScript avec React et construit avec Vite ; la mise en forme repose sur Tailwind CSS, les animations sur Motion et Three.js, les tests sur Vitest. Aucune bibliothèque ne touche aux données chiffrées : toute la cryptographie passe par l'API Web Crypto. Nous avons écrit environ 6 000 lignes de TypeScript, dont un moteur cryptographique isolé de l'interface.

Tableau 5 : Modules du moteur cryptographique

Module	Lignes	Rôle
keys.ts	280	Génération, export et import des clés ; protection de la clé privée
hybrid.ts	272	Chiffrement et déchiffrement hybrides, morceau par morceau
container.ts	91	Lecture et écriture de l'en-tête .rsaenc
encoding.ts, errors.ts	141	Conversions, empreintes, erreurs typées en français
textbook.ts	140	RSA programmé à la main, réservé à la page pédagogique
selftest.ts	300	Dix-neuf cas de test partagés entre Node.js et le navigateur

5.2. Interface

Le site compte sept écrans : Accueil, Chiffrer, Déchiffrer, Clés RSA, Comprendre RSA, Tests et À propos, aux couleurs de la maquette du binôme. Les animations ne sont pas décoratives : un module reçoit l'avancement réel du calcul, et une étape n'apparaît terminée que lorsqu'elle l'est. En mode démonstration, chaque étape reste visible au moins 950 millisecondes, le temps de la commenter.



Figure 6 : Page d'accueil de l'application

Lors de la création d'une paire de clés, le site affiche les valeurs réellement calculées, secrets masqués, puis la carte de la clé avec l'empreinte à lire à voix haute.



Figure 7 : Valeurs calculées lors de la génération d'une clé

Pendant un chiffrement, le panneau « Ce qui se passe » affiche les quatre étapes réelles et une grille représentant les 384 premiers octets du fichier, qui perd progressivement toute structure. Au déchiffrement, le chemin inverse s'affiche et les deux empreintes SHA-256 sont comparées.

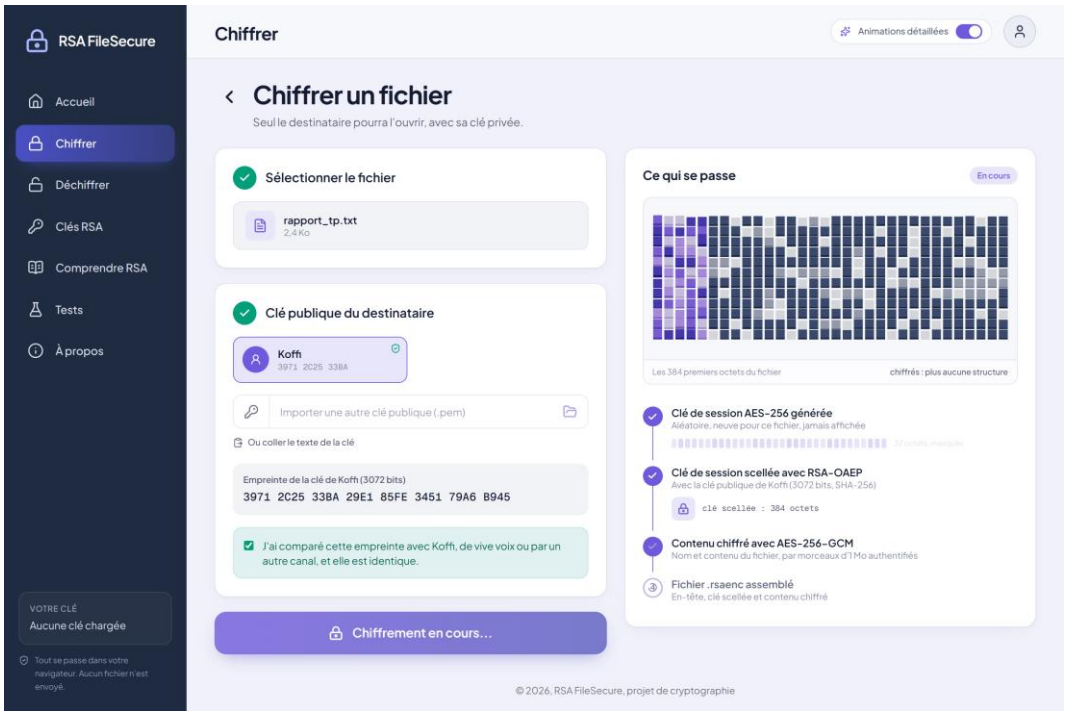


Figure 8 : Écran de chiffrement pendant le calcul



Figure 9 : Déchiffrement réussi, empreintes identiques

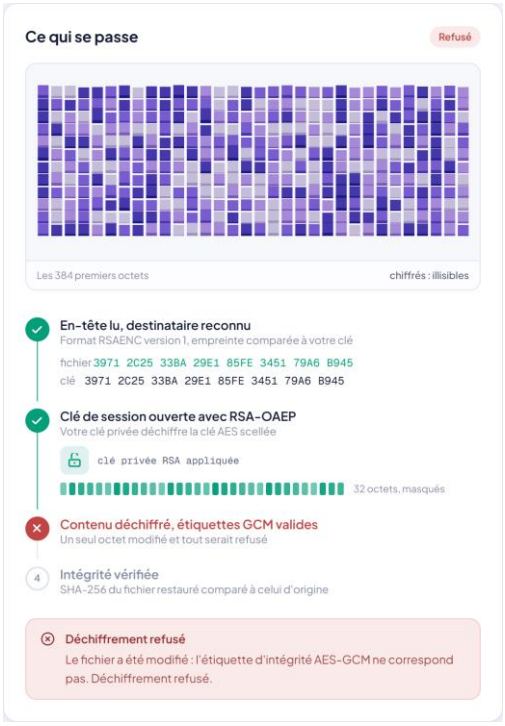


Figure 10 : Refus d'un fichier modifié d'un seul octet

5.3. Sécurité et déploiement

La clé privée n'est jamais enregistrée : une fois importée depuis le fichier protégé, elle est déclarée non exportable et disparaît à la fermeture de l'onglet. Seules les clés publiques des

binômes sont mémorisées, avec la mention « empreinte vérifiée », obligatoire avant tout chiffrement. La version publiée embarque une politique de sécurité du contenu dont la directive connect-src 'none' interdit toute requête réseau après le chargement.

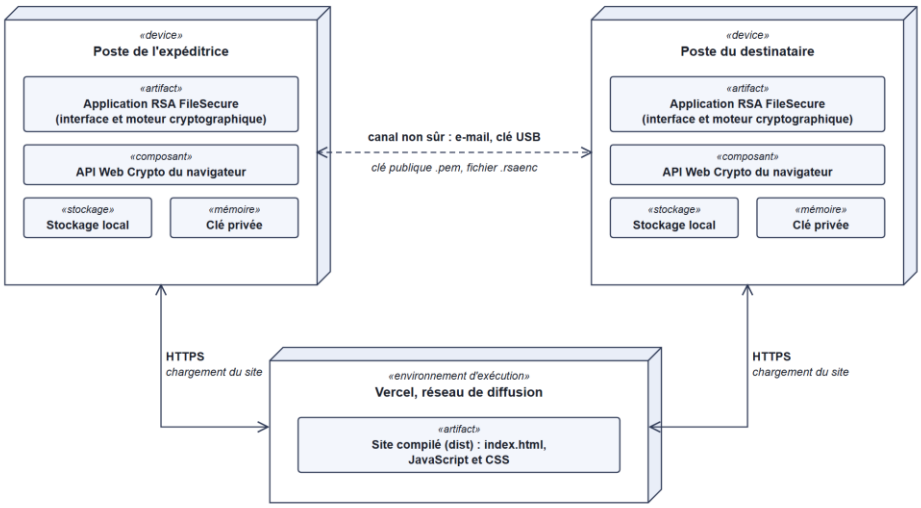


Figure 11 : Diagramme de déploiement de l'application

Le site est déployé sur Vercel, à l'adresse https://rsa-filesecure.vercel.app, en HTTPS comme l'exige l'API Web Crypto. Seules les sources sont transmises ; Vercel compile le projet et ajoute des en-têtes de sécurité HTTP. Les dix-neuf tests de la page « Tests » passent sur le site public.

6. Tests et résultats

6.1. Tests automatisés

Les vingt-deux tests automatisés passent en 1,65 seconde. Les dix-neuf tests du moteur sont écrits une seule fois et exécutés à deux endroits : par Vitest dans Node.js, et par la page « Tests » dans le navigateur, sous les yeux du jury.

Tableau 6 : Répartition des tests automatisés

Famille	Tests	Ce qui est vérifié
Clés	6	Génération, formats PEM, mot de passe correct et incorrect, refus d'une clé privée et d'une clé de 1024 bits
Chiffrement	5	Texte, fichier de 300 Ko en plusieurs morceaux, fichier vide, deux chiffrements différents, limite de 100 Mo
Refus à la lecture	6	Octet modifié n'importe où, autre clé, en-tête falsifié, autre format, troncature, réordonnancement
RSA pédagogique	2	Exemple p = 61, q = 53 ; clé de 48 bits cassée par factorisation

Famille	Tests	Ce qui est vérifié
Interopérabilité	3	OpenSSL chiffre pour notre clé, ouvre notre clé de session, et sa paire fonctionne dans notre moteur

Les trois tests d'interopérabilité s'appuient sur le module crypto de Node.js, qui repose sur OpenSSL : nos formats sont standards, et non une construction propre au projet.

6.2. Performances

Les mesures ont été prises dans Chrome 156, sur un ordinateur portable équipé d'un Intel Core i7-8565U et de 16 Go de mémoire. Elles portent sur le calcul seul. Une clé de 3072 bits se génère en 0,35 seconde en médiane et en moins de 0,75 seconde dans le pire des 21 essais ; la protection par mot de passe prend 0,45 seconde.

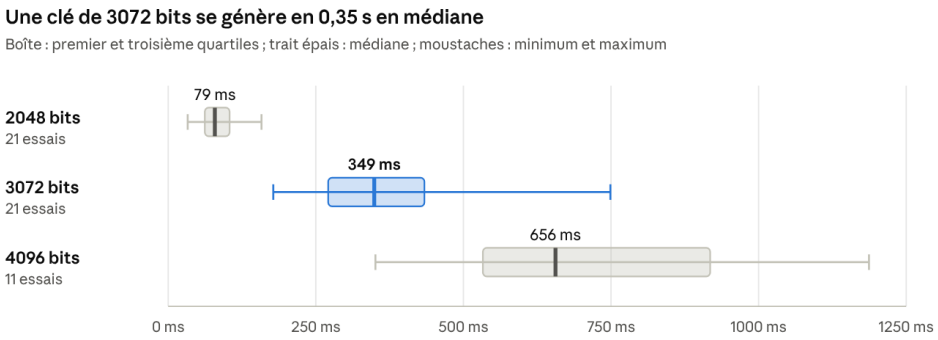


Figure 12 : Durée de génération d'une paire de clés selon sa taille

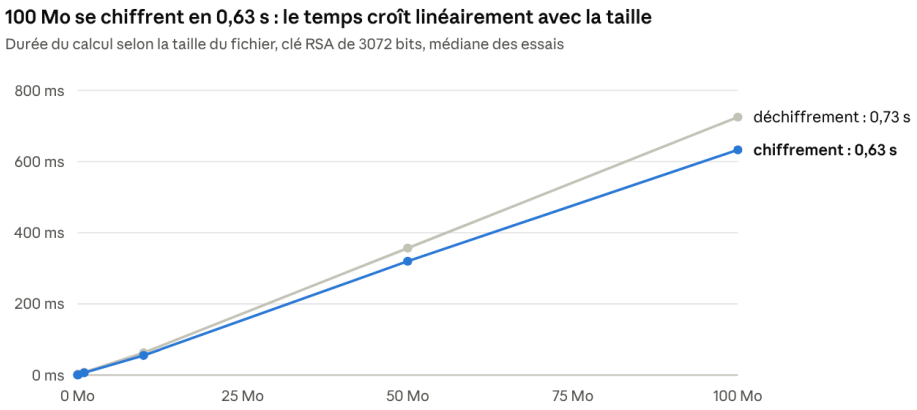


Figure 13 : Durée de chiffrement et de déchiffrement selon la taille du fichier

Le temps croît linéairement avec la taille : 100 Mo se chiffrent en 0,63 seconde et se déchiffrent en 0,73 seconde, soit environ 160 et 140 Mo par seconde. Le fichier chiffré ne grossit que de 894 octets pour 1 Ko et de 2 501 octets pour 100 Mo.

7. Difficultés rencontrées

La principale difficulté était conceptuelle : concilier l'énoncé, qui demande RSA, avec la réalité de RSA, qui ne chiffre que quelques centaines d'octets. Le calcul de la taille maximale d'un message et du temps nécessaire pour 100 Mo nous a convaincus d'adopter le chiffrement hybride. Les autres difficultés sont apparues pendant la réalisation.

Tableau 7 : Difficultés rencontrées et solutions retenues

Difficulté	Cause	Solution retenue
Découper sans affaiblir GCM	Un IV réutilisé ou un morceau supprimé passeraient inaperçus	IV unique par morceau ; numéro et drapeau « dernier » authentifiés
Animations figées en développement	Le mode strict de React monte chaque composant deux fois	Indicateur d'activité réinitialisé à chaque montage
Étapes bloquées fenêtre masquée	Le navigateur suspend les rafraîchissements d'écran	Minuterie de secours de 120 ms
Polices bloquées en production	La politique de sécurité refuse les ressources intégrées	Ressources servies comme fichiers
Débordements sur téléphone	Empreintes et grilles trop larges pour 375 pixels	Retour à la ligne et grilles sur une colonne

8. Organisation du travail

Nous avons découpé le projet en cinq lots, chacun relu par l'autre membre du binôme avant d'être considéré comme terminé. Pour le moteur cryptographique, cette relecture portait d'abord sur les tests de refus.

Tableau 8 : Lots de travail et livrables

Lot	Contenu	Livrable
1. Analyse et conception	Sujet, choix cryptographiques, format .rsaenc, maquette	Plan détaillé et maquette
2. Moteur cryptographique	Clés, chiffrement hybride, formats, RSA pédagogique	Dossier src/crypto
3. Interface	Sept écrans, animations, version téléphone	Site complet, publié en ligne
4. Tests	Cas partagés, interopérabilité OpenSSL, mesures	22 tests automatisés
5. Documentation	Guide d'utilisation, vidéo, rapport	Guide PDF, vidéo narrée, ce rapport

Pour les démonstrations, chacun joue un rôle fixe, l'un Koffi, l'autre Awa, sur son propre ordinateur. Cette répartition oblige à faire passer la clé publique et le fichier chiffré par un vrai canal.

9. Limites et perspectives

L'application ne prouve pas l'identité de l'expéditrice : toute personne qui possède la clé publique de Koffi peut lui envoyer un fichier valide. Sa sécurité dépend aussi du code distribué par l'hébergeur ; lancer le site en local écarte ce risque. Le fichier chiffré laisse deviner la taille de l'original et l'identifiant de la clé du destinataire. Enfin, le fichier est traité en mémoire, d'où la limite de 100 Mo, et PBKDF2 ralentit une attaque par dictionnaire sans l'empêcher.

Plusieurs évolutions prolongeraient ce travail :

- faire signer chaque fichier par l'expéditrice avec sa propre clé privée (RSA-PSS), pour prouver son identité ;
- accepter plusieurs destinataires, en scellant la même clé de session pour chaque clé publique ;
- chiffrer en flux, ce que le format par morceaux permet déjà, pour lever la limite de 100 Mo ;
- remplacer PBKDF2 par Argon2, compilé en WebAssembly, plus résistant aux attaques sur carte graphique.

10. Conclusion

RSA FileSecure répond à l'énoncé : Awa chiffre un fichier pour Koffi, et seul Koffi peut le déchiffrer, grâce à sa clé privée RSA. L'application va au-delà de la démonstration : publiée en ligne, elle traite 100 Mo en moins d'une seconde, détecte toute modification et n'envoie aucune donnée sur le réseau. Vingt-deux tests automatisés, dont trois avec OpenSSL, en vérifient le fonctionnement.

Ce projet nous a surtout appris qu'un algorithme correct ne suffit pas. RSA de manuel est mathématiquement juste, mais inutilisable tel quel ; la sécurité vient de l'assemblage, du remplissage OAEP à la vérification des empreintes, chaque détail répondant à une attaque connue. La signature des fichiers constitue la prochaine étape.

Références

Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM, 21*(2), 120-126.

Moriarty, K., Kaliski, B., Jonsson, J., & Rusch, A. (2016). *PKCS #1: RSA cryptography specifications version 2.2* (RFC 8017). https://www.rfc-editor.org/rfc/rfc8017

Moriarty, K., Kaliski, B., & Rusch, A. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). https://www.rfc-editor.org/rfc/rfc8018

National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS 197).

Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST SP 800-38D). NIST.

Barker, E. (2020). *Recommendation for key management: Part 1 – General* (NIST SP 800-57 Part 1 Rev. 5). NIST.

OWASP Foundation. (s.d.). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

World Wide Web Consortium. (2017). *Web Cryptography API*. https://www.w3.org/TR/WebCryptoAPI/

Annexe A : liste des tests automatisés

Tableau 9 : Liste des tests automatisés

N°	Test	Résultat attendu
1	Génération d'une paire RSA	Deux clés de 2048 bits, empreinte de 64 caractères
2	Clé publique : export puis import PEM	Même empreinte et même nom
3	Clé privée protégée : bon mot de passe	La clé relue déchiffre
4	Clé privée protégée : mauvais mot de passe	Refus
5	Chiffrer puis déchiffrer un texte	Contenu et SHA-256 identiques
6	Fichier de 300 Ko en plusieurs morceaux	Identique octet par octet
7	Fichier vide	Chiffré et restauré sans erreur
8	Un octet modifié n'importe où	Refus aux quatre positions
9	Clé d'une autre personne	Refus
10	Mauvaise clé, en-tête falsifié	La clé de session ne s'ouvre pas
11	Fichier qui n'est pas un .rsaenc	Fichier non reconnu
12	Fichier tronqué	Refus
13	Morceaux intervertis	Refus
14	Deux chiffrements du même fichier	Résultats différents
15	Clé privée à la place d'une clé publique	Refus avec avertissement
16	Clé de 1024 bits	Refus : trop faible
17	Fichier de plus de 100 Mo	Refus clair
18	RSA à la main, p = 61, q = 53	65 chiffré en 2790 puis retrouvé
19	Attaque d'une clé de 48 bits	Clé privée retrouvée
20	OpenSSL chiffre pour notre clé PEM	Web Crypto déchiffre
21	OpenSSL ouvre la clé de session	32 octets retrouvés
22	Paire OpenSSL de 3072 bits	Fichier restauré à l'identique

Annexe B : structure du fichier .rsaenc

Tableau 10 : Structure du fichier chiffré .rsaenc

Position	Taille	Contenu
0	6 octets	« RSAENC » en ASCII : 52 53 41 45 4E 43
6	1 octet	Version du format : 01
7	4 octets	Longueur L de l'en-tête JSON, entier gros-boutiste
11	L octets	En-tête : alg, kid, bits, wrappedKey, ivPrefix, chunkSize, chunks
11 + L	Reste	Morceaux chiffrés par AES-GCM, chacun suivi de son étiquette de 16 octets

Voici l'en-tête réel d'un fichier chiffré pour une clé de 3072 bits, la clé scellée étant raccourcie :

```
{
  "alg": { "key": "RSA-OAEP-256", "data": "AES-256-GCM" },
  "kid": "48c86ea712c8313e26c18dfaedeb93d33deb1fbabbecdf75d6ea121c8b36b62a",
  "bits": 3072,
  "wrappedKey":
"IhqSoxG8lHXLW1txCRVimoEQ...BStalUdaYQz1QvTy1mAmDyOR9pJwDsqvkSwhPeuZ/YtCQ",
  "ivPrefix": "qnd4XIHabzk=",
  "chunkSize": 1048576,
  "chunks": 1
}
```

Annexe C : vérification avec OpenSSL

Ces commandes, valables avec OpenSSL 3, permettent de contrôler nos clés sans passer par le site : la première affiche la taille et le module de la clé publique, la seconde chiffre un secret pour le destinataire avec le même remplissage que l'application.

```
openssl pkey -pubin -in cle_publique_koffi.pem -text -noout

openssl pkeyutl -encrypt -pubin -inkey cle_publique_koffi.pem \
  -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256 \
  -pkeyopt rsa_mgf1_md:sha256 -in secret.bin -out secret.enc
```